

Projet d'intégration & de développement « my wardrobe »

Rapport remis par Moise Mundaya

3e année informatique

Table des matières

Rappel du sujet	2
Spécifications des choix.....	2
Justification technique	3
Expo + React native	3
Typescript	3
Système de routage avec expo-router	3
React-hook-form + zod	3
Expo-image-picker	3
Supabase	3
Architecture	4
Partie navigation (app)	4
Partie application (src)	4
Partie commune (shared).....	5
Flux principaux.....	5
Flux création de profil	5
Auto-évaluation	6
Conclusion	6

Rappel du sujet

My Wardrobe est une plateforme sociale dédiée à la gestion et au partage de vêtements et d'accessoires. L'application permet aux utilisateurs de centraliser virtuellement leurs pièces afin d'avoir leur garde-robe à portée de main et ainsi composer leurs tenues facilement. Au-delà de l'aspect pratique, le projet intègre une dimension communautaire : les utilisateurs peuvent partager leur garde-robe avec leurs proches pour s'inspirer mutuellement, échanger des idées de looks et interagir avec les publications de leur réseau.

Spécifications des choix

Fonctionnalités	Description
Authentification	Permettre à un utilisateur de créer un compte via son adresse e-mail et d'enregistrer un mot de passe afin de pouvoir se connecter par la suite
Gestion du profil utilisateur	L'utilisateur pourra créer un profil, le consulter, le modifier et le supprimer.
Gestion des fiches vêtements	Création, visualisation, modification et suppression de la fiche d'un vêtement
Gestion de la caméra	Permettre à l'application d'accéder à l'appareil photo de l'utilisateur afin d'en prendre une depuis l'application, ou d'accéder à sa galerie pour sélectionner une image existante.
Engagement social	L'utilisateur a la possibilité d'aimer, et de commenter les publications de son réseau.
Favoris	Possibilité d'ajouter ou de retirer un post de ses favoris pour le retrouver facilement plus tard.
Feed d'actualités	L'utilisateur a la possibilité de voir et d'interagir avec les publications de ses amis, ainsi que les siennes, via un espace dédié.
Gestion d'amitié	Possibilité d'envoyer une demande à un utilisateur, d'accepter ou refuser une demande reçue. L'utilisateur peut aussi voir la liste de ses amis ainsi que leur bio
Recherche d'utilisateur	L'utilisateur a la possibilité d'en rechercher un autre via son nom d'utilisateur.
Système de notation	Un compteur de likes, intégré à chaque publication, s'incrémente automatiquement à chaque mention « J'aime » reçue.

Justification technique

Expo + React native

React Native et son écosystème Expo ont été choisis comme technologies principales pour ce projet. C'est un choix technique judicieux, car il permet d'écrire le code source une seule fois tout en garantissant des performances optimales. En effet, le framework se charge de traduire ce code en exploitant les composants natifs propres à chaque système d'exploitation (iOS et Android).

Expo, quant à lui, simplifie considérablement le processus de développement grâce à divers outils intégrés. Il offre notamment la possibilité de tester l'application en temps réel sur des appareils physiques (via Expo Go), facilite l'accès aux fonctionnalités matérielles du téléphone (caméra, galerie, notifications) sans nécessiter de configuration complexe en code natif.

Typescript

Ce langage de programmation apporte une bien meilleure détection des erreurs que JavaScript. De plus, grâce à son typage statique, il offre **auto-complétion optimale** et garantit une **bonne** maintenabilité du code.

Système de routage avec expo-router

Étant natif à Expo, ce système de routage s'intègre naturellement. L'ensemble de la navigation est orchestré par des *layouts*, ce qui permet de garder une architecture simple et claire.

React-hook-form + zod

Ce couple de technologies a été choisi pour son excellente synergie. React Hook Form prend en charge la gestion globale du formulaire, facilitant le contrôle des saisies, des états, des erreurs ainsi que la soumission.

Zod, quant à lui, agit comme un validateur robuste. Il s'assure que les types sont exacts et que les valeurs entrées respectent les règles de validation. Toutes ces règles sont centralisées dans un schéma qui peut être directement converti en type TypeScript, synchronisant ainsi parfaitement le formulaire et la validation. De plus, un schéma Zod est entièrement réutilisable et peut même être étendu selon les besoins.

Expo-image-picker

Cet outil appartenant à l'écosystème Expo, c'est donc vers ce dernier que je me suis tourné pour gérer l'accès à la caméra de l'utilisateur.

Supabase

Afin de pouvoir me concentrer pleinement sur le développement front-end tout en livrant une application fonctionnelle, j'ai choisi d'utiliser Supabase. Cette plateforme open source permet de déployer rapidement une infrastructure back-end complète.

Supabase simplifie la gestion de la base de données, de l'authentification et la génération automatique des appels API grâce à ses outils intégrés. Elle offre notamment un module "Storage" facilitant le stockage des photos, et garantit la sécurité des données en implémentant un système de "policies" (RLS), ce qui permet de définir des permissions d'accès précises au niveau de chaque table de la base de données.

Architecture

J'ai opté pour une architecture dite « *feature-based* ». Axée sur les fonctionnalités, cette approche sépare le projet en plusieurs modules indépendants. Chacun de ces groupes de fonctionnalités possède ainsi ses propres éléments (composants, logique), tout en s'appuyant sur des ressources et des composants partagés.

Partie navigation (app)

Cette partie centralise la navigation de l'application à travers l'utilisation des *layouts*. Ces fichiers ne contiennent aucune logique métier : ils servent exclusivement à orchestrer la navigation entre les écrans liés à l'inscription/ connexion de l'utilisateur (authentification), la barre de navigation (*tabs*) et les écrans principaux de l'application (*friends, user, clothes*).

Partie application (src)

Ce répertoire centralise tous les modules liés aux fonctionnalités au sein d'un dossier *features*, ainsi que le dossier *utils* (qui gère notamment l'initialisation du client Supabase).

Le dossier *features* isole le code en fonction des différentes fonctionnalités de l'application :

1) Auth (authentication)

Ce module gère les formulaires d'inscription et de connexion, la logique de redirection conditionnelle basée sur l'état d'authentification de l'utilisateur, ainsi que les écrans d'affichage associés.

2) Camera

Ce dossier regroupe l'ensemble de la logique relative à l'utilisation de l'appareil photo de l'utilisateur. Lors de la prise en photo d'un nouveau vêtement par exemple, c'est ce module qui est sollicité pour fournir les méthodes adéquates.

3) Clothes

Ce module centralise tout ce qui est relatif aux vêtements, ce qui en fait le dossier le plus volumineux du projet. Il regroupe les différents composants d'interface, tels que la fiche du vêtement (*ClothesCard*), ainsi que les *hooks* gérant l'engagement social (likes, commentaires, favoris). On y retrouve également les écrans et les services qui y sont dédiés, sans oublier les *mappers*, essentiels pour assurer la correspondance entre les types front-end et le schéma de la base de données.

4) Friend

Cette partie est entièrement dédiée à la gestion des relations d'amitié entre les utilisateurs. Il intègre la logique et les méthodes liées au cycle de vie des demandes d'ami (envoi, réception, acceptation et refus). On y retrouve également la fonctionnalité de recherche de profils par nom d'utilisateur (username), ainsi que les écrans d'interface permettant de centraliser et de visualiser l'ensemble de ces interactions.

5) User

Ce module centralise tout ce qui concerne la gestion du profil de l'utilisateur. On y retrouve les différents écrans liés à la création du profil après inscription, à l'affichage de l'espace personnel et à l'édition des informations. Il intègre l'ensemble de la logique métier gérant les interactions avec la base de données (création du *username* et de la bio, lecture, mise à jour et suppression des données du compte). Enfin, ce dossier regroupe les composants d'interface dédiés (en-têtes personnalisés, cartes d'informations), les *hooks* permettant de gérer le cycle de vie de l'interface (rafraîchissement

des données, déconnexion, édition) ainsi que les schémas de validation des formulaires garantissant le respect des contraintes de saisie.

Partie commune (shared)

Ce dossier regroupe les éléments communs à plusieurs fonctionnalités afin d'éviter toute duplication de code. Il contient principalement les modèles de données globaux (shared/model), définissant les types et interfaces du projet, ainsi que les composants UI réutilisables (shared/components). C'est la véritable bibliothèque de briques génériques de l'application.

Flux principaux

Flux création de profil

Le parcours de création d'un profil s'articule de la manière suivante :

- **Inscription** : L'utilisateur s'inscrit d'abord via l'écran d'authentification. Dès que la session est créée avec succès, il est automatiquement redirigé vers l'écran de configuration de profil.
- **Saisie des informations** : L'utilisateur est invité à remplir un formulaire (comprenant notamment son nom d'utilisateur et une biographie). Les saisies sont contrôlées en temps réel grâce aux schémas de validation (Zod).
- **Enregistrement** : Lors de la soumission du formulaire, l'application récupère l'identifiant de l'utilisateur connecté via le service d'authentification de Supabase, puis procède à l'écriture des données dans la table correspondante.
- **Gestion des retours** : Si la création réussit, une alerte de confirmation s'affiche et l'utilisateur est redirigé vers l'onglet de découverte. En cas d'erreur, un message explicite est directement affiché sur l'écran de profil.

Flux de publication d'un vêtement

- **Saisie et Image** : Depuis l'onglet de publication, l'utilisateur remplit un formulaire validé (nom, catégorie, visibilité) et y associe une photo (URL, caméra ou galerie).
- **Traitement et Stockage** : À la validation, l'utilisateur connecté est identifié. Si l'image provient du téléphone, elle est directement téléversée et sécurisée dans le Storage de Supabase.
- **Insertion en base** : Les données du vêtement sont sauvegardées en base de données, puis formatées (mapping) pour être exploitables par l'interface front-end.
- **Mise à jour de l'interface** : Le vêtement est instantanément ajouté au profil dans "Mon dressing" et, s'il est public, apparaît dans le flux global "Discover".

Auto-évaluation

Parmi les fonctionnalités initialement prévues, certaines n'ont pas encore été finalisées, tandis que d'autres n'ont pas pu être implémentées lors de cette première phase de développement.

Fonctionnalités	Commentaire
CRUD tenues	Faute de temps, la fonctionnalité permettant à l'utilisateur de composer des ensembles (vêtements et accessoires) n'a pas pu être finalisée.
Commentaire	Actuellement, l'utilisateur a la possibilité de publier un commentaire, mais la fonctionnalité permettant de le supprimer n'a pas été développée. Il s'agit d'un manque d'attention de ma part.
Design et UI	Ayant concentré mes efforts sur le développement technique et le bon fonctionnement de l'application, l'aspect visuel et le design de l'interface ont été mis au second plan pour le moment.
Nommage	Certains noms de méthodes et de dossiers manquent de précision.

Conclusion

Ce projet m'a permis de consolider mes compétences en développement front-end, tout particulièrement avec le framework React Native. L'implémentation de formulaires et leur validation avec Zod m'ont appris à garantir la fiabilité et la sécurité des données avant leur envoi vers le back-end. J'ai également pu constater les avantages d'une architecture logicielle modulaire (*feature-based*), où l'organisation par fonctionnalités est priorisée. Côté back-end, la découverte et la prise en main de Supabase m'ont fait réaliser l'utilité de cette plateforme dans un projet concret. Même si l'application n'est pas totalement finalisée faute de temps, cette réalisation s'est révélée être une expérience formatrice et enrichissante.